

Gear&Glitch Project User Manual

1. Project Overview

- Gear&Glitch is a complete multi-branch sales & management system (SaaS) combining:
- A shopper storefront with product catalog, shopping cart, wishlist, quotes, and two-step checkout.
 - A customer repair portal for booking and tracking repair tickets.
 - A unified, role-aware **staff portal** for repair ticket management, stock control, catalog, team, finance, plans, providers, settings, and business oversight.
 - A **control plane** operator dashboard that provisions and manages every client instance.

It uses a **Next.js 14 (Pages Router) + TypeScript** frontend, a **Node.js + Express + TypeScript** backend, and **PostgreSQL** for persistence (one database per client, provisioned on Neon). Multi-tenancy is handled by the control plane, which spins up an isolated Neon DB + Render backend + Vercel frontend for each client.

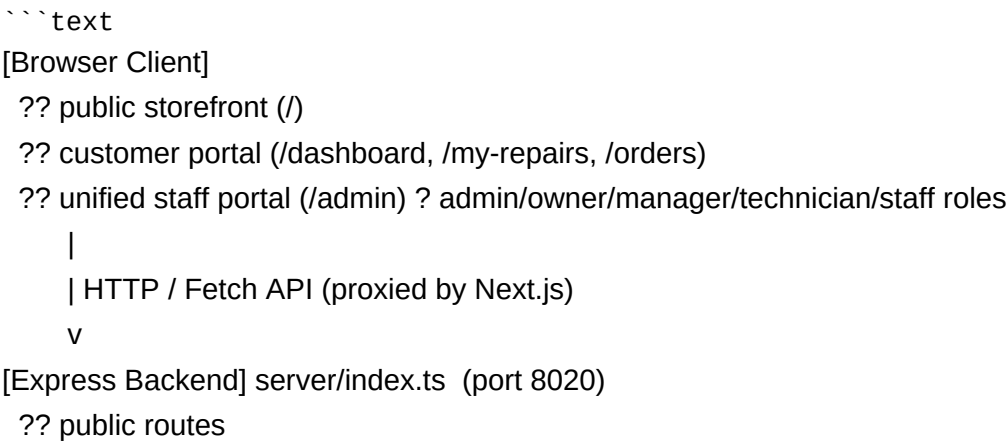
Languages & tools: TypeScript throughout (frontend, backend, control plane), with JavaScript for the control-plane dashboard and Next.js pages, HTML5/CSS3 (design-token system + GPU-accelerated CSS animations) for the UI, and SQL (PostgreSQL DDL/migrations) for data. Infrastructure: Neon (PostgreSQL), Render (backends + control plane), Vercel (storefronts), Cloudinary (images), Cloudflare (DNS).

2. Architecture

Three layers, cleanly separated:

Layer	Tech	Port	Purpose
Frontend	Next.js 14 (Pages Router) + TypeScript	3000	UI rendering, client-side routing
Backend	Express + TypeScript	8020	REST API, JWT auth, Cloudinary uploads, M-Pesa, PDF generation
Database	PostgreSQL via `pg` Pool	?	Cloud database (Neon), one schema set per client
Control plane	Express + TypeScript	4000	Central operator dashboard (separate app at `control-plane/`)

Runtime architecture:



?? customer routes
?? staff/admin/owner routes
?? provider routes
?? control-plane management routes (x-control-plane-key auth)

|
v

[PostgreSQL] (per-client Neon database)

...

****Multi-tenant provisioning:****

```text

[Control Plane] control-plane/ (port 4000, own Neon DB)

?? Add Client ? creates:

? ?? Neon database (client DB)

? ?? Render web service (client backend, port 8020)

? ?? Vercel project (client frontend, rootDirectory: "frontend")

?? per-client CONTROL\_PLANE\_SECRET for remote management

?? health monitoring (every 5 min), Slack alerts

?? per-client feature overrides pushed to each backend

...

---

### 3. Folder Structure

...

Laptop sale/

??? frontend/ # Next.js 14 (Pages Router + TypeScript)

? ??? components/

? ? ??? ui/ # Reusable component library (Button, Card, Input, Modal, Badge...)

? ? ??? admin/ # Admin/Owner shared components (AdminProducts, ProvidersPage, ...)

? ? ??? Layout.tsx # Responsive header (default + springboard modes)

? ? ??? ProductCard.tsx # Product card with strikethrough sale price

? ? ??? ... # MarqueeBanner, Toast, Skeleton, LoadingScreen, ScrollReveal...

? ??? lib/

? ? ??? api.ts # API client with token management

? ? ??? app-context.tsx # React context ? auth, theme, cart, settings, favicon/tab title

? ? ??? types.ts # TypeScript interfaces

? ? ??? features.ts # useFeature() hook for subscription feature gating

? ??? layouts/ # Storefront themes + runtime layout engine

? ? ??? original.tsx # Original theme (premium marketing hero)

? ? ??? amazon.tsx, jumia.tsx, mobile.tsx, custom.tsx

? ? ??? dynamic-engine.tsx # Generic JSON layout renderer

? ? ??? index.tsx # Layout registry + provider

? ??? pages/

? ? ??? index.tsx # Homepage

```

? ? ??? admin.tsx # Unified role-aware staff portal (replaces /admin, /owner, /backoffice)
? ? ??? login.tsx, dashboard.tsx, cart.tsx, orders.tsx, ...
? ? ??? _document.tsx # Static <title> placeholder (replaced by store name at runtime)
? ??? styles/ # globals.css (design tokens), animations.css
??? server/ # Express backend (TypeScript)
? ??? index.ts # All API routes (asyncHandler-wrapped)
? ??? db.ts # PostgreSQL layer (migrations, seed, settings)
? ??? schema.sql # PostgreSQL schema
? ??? auth.ts # JWT auth + login/register
? ??? repairs.ts # Repair ticket lifecycle
? ??? permissions.ts # Role-based permissions
? ??? upload.ts # Multer image upload (Cloudinary production / disk dev)
? ??? mpesa.ts # Daraja API STK Push
? ??? email.ts # Unified email transporter
? ??? notify.ts # Email notification helpers
? ??? routes/shared.ts # asyncHandler, escapeHtml, validation helpers, invoice templates
??? control-plane/ # Operator dashboard (Express + own Neon DB)
? ??? server/ # index.ts, provision.ts (Neon/Render/Vercel/Cloudflare), db.ts
? ??? public/ # Static dashboard (HTML/JS) incl. feature-override picker
??? render.yaml # Render.com deployment config
??? vercel.json # Vercel deployment config
??? README.md # Main project documentation
` ``

```

---

## 4. How the Application Starts

### Local setup

1. Install PostgreSQL and create a database (`laptop\_sale`).
2. `npm install` at the root, then `cd frontend && npm install`.
3. `copy .env.example .env` and set `DATABASE\_URL`, `JWT\_SECRET`, `ADMIN\_PASSWORD`, `TECH\_PASSWORD`.
4. Start everything: `npm run dev:all`
  - Backend: `http://localhost:8020`
  - Frontend: `http://localhost:3000`

See `LOCAL\_SETUP.md` for the detailed step-by-step guide.

### Default URLs (dev)

- `http://localhost:3000` ? storefront homepage.
- `http://localhost:3000/login` ? unified login (customer / staff / provider; Google Sign-In supported).
- `http://localhost:3000/admin` ? unified staff portal (admin / owner / manager / technician / staff).
- `http://localhost:4000` ? control plane dashboard (separate app).

---

## 5. Main User Roles

### Customer

- Browse products, search, filter by category/subcategory.
- Add to cart, wishlist, request quotes.
- Two-step checkout with delivery details (47 Kenyan counties) and M-Pesa payment.
- Track orders, download invoices (shipped/delivered), book and track repairs, message providers, review products (1-5 stars).

### Staff (technician / manager / role-based)

- Log in to the unified staff portal at `/admin`.
- Manage repair tickets (assign, status, notes, parts, cost estimates sent to customers).
- What the sidebar shows depends on the user's effective permissions (base role + assigned custom roles + direct per-user permissions): technicians and staff see Dashboard + Repairs; managers additionally get Products, Orders, Customers, Quotations, Invoices, Credit Notes, Reports, Messages, and Stock; owners get business oversight on top.

### Provider

- Log in to the unified staff portal at `/admin` (provider role, added by an admin from **Users & Permissions**).
- What the provider sees is driven by their assigned permissions (e.g. repairs, products, stock, calendar, order viewing, messaging). External provider accounts without staff permissions no longer have a separate portal.

### Admin

- Superuser in `/admin`: every section ? products, categories, orders, staff, roles, plans, providers, invoices, credit notes, coupons, reports, stock, branches, clients, splashes, reviews, messages, settings, storefront layouts.

### Owner

- Business oversight in `/admin` (owner role): dashboard, products, customers, messages, quotes, reports, stock control, tech repairs, about us, storefront, shop subscription, audit log. Everything except Users, Roles, Plans, Settings, Storefront, and Purchases. Feature-gated by subscription plan.

### Control Plane Operator (Gear&Glitch team)

- Log in to the control plane dashboard: provision clients, monitor health, manage plans/subscriptions/invoices, deploy updates, run backups, configure SMTP/Cloudinary, and fine-tune per-client features.

---

## 6. Key Panels

## Staff Portal (`/admin`)

One unified, role-aware portal replaces the former separate `/admin`, `/owner`, and `/backoffice` pages. The sidebar shows exactly what the signed-in user's permissions allow; feature gating hides sections the active plan doesn't include. Sections include: Dashboard, Products, **\*\*Groups\*\***, Categories, **\*\*Category Order\*\***, **\*\*Repairs\*\*** (Services: Tickets, Calendar, Page Content), Coupons, Gift Cards, Campaigns, Abandoned Carts, Orders, Users & Permissions, Roles, Plans, Providers, Invoices, Credit Notes, Reports (5 sub-tabs), Stock on Hand, Stock Transfers, Stock Take, Purchases, Suppliers, Branches, Clients, Messages, Reviews, Spec Templates, Splashes, About Us, Storefront, Shop Subscription, Settings.

Access is fine-tuned in **\*\*Users & Permissions\*\*** (assign/remove roles per user, toggle direct permissions) and **\*\*Roles\*\*** (define custom roles with granular permission toggles ? every sidebar view maps to a permission such as `order:view`, `customer:view`, `invoice:view`, `reports:view`, `stock:list`, or `messaging:view`).

## Customer management (Admin ? Customers)

- **\*\*Edit a customer\*\*** ? the **\*\*Edit\*\*** button on any customer row opens an inline panel to update name, email, and phone, or set a new password (leave blank to keep the current password). A duplicate email is rejected with a clear error.
- **\*\*Delete a customer\*\*** ? deleting a customer also removes all their dependent records in one transaction: repair tickets (with their parts, updates, and images), messages, product reviews, loyalty points and transactions, coupon usage, credit notes, eTIMS sales records, orders (with items, invoices, gift-card redemptions, refunds), carts, wishlist, quotes, and cart-recovery reminders. Every customer can be deleted regardless of history ? but deletion is permanent and cannot be undone.

## Control Plane Dashboard

Clients, Plans, Changelog, Deploy Log, Backups, Settings (SMTP/Cloudinary), Audit Log, Users (with 2FA), plus a **\*\*notification bell\*\*** (unread badge + dropdown) and a **\*\*payment-reminders banner\*\*** on the Clients tab.

---

## 7. Storefront

### Layouts

Five built-in layout themes ? Original, Amazon, Jumia, Mobile, Custom ? plus a runtime JSON layout builder for admin-created dynamic themes. The active layout is stored in the `storefront\_layouts` table and selected from Admin ? Settings ? Storefront.

### Product Groups

Managed collections built from the old free-text category groups (`product\_groups` table). Admins create/edit/delete groups and toggle each one **\*\*active\*\*** from Admin ? Groups. Active groups get public `/groups` (index) and `/group/[slug]` (product grid) pages, and appear as filters in the Sales Report and Stock Summary. Products assign to a group via the product form's Group dropdown; category remains optional. Existing category group names were migrated into group rows automatically, and `products.group\_id` was backfilled from each product's category.

## Repairs page content

The public `/repairs`` page is fully editable from **Admin ? Services ? Repairs ? Page Content**: change the intro paragraph and add/remove/edit the service panels (title + description) customers see. Each panel can be hidden without deleting it (untick **Show on the /repairs page**) or turned into a **booking button**: tick **Booking button**, then set a repair type (for automatic base pricing), a device type to pre-fill, a default issue description, and symptoms to pre-check. Clicking that panel on the storefront opens `/repair-book`` with those fields already filled ? the customer just confirms and submits, and the ticket is created with the linked repair type so a quote is calculated immediately. Content is saved in settings and served to the storefront via `GET /api/repairs-page`` (falls back to the default panels if nothing has been saved yet).

## Header navigation & hero chips stay in sync with categories

The storefront header nav buttons (next to the Sign in button) and the hero's category chips are reconciled against the live category list on every load ? no manual "sync" step needed:

- **Deleted categories** are automatically removed from the header nav and hero chips.
- **Newly added categories** are automatically appended to both.
- Static page links (Repairs, Cart, Wishlist, About Us, Contact) are preserved, as is the admin-configured nav order for existing items.
- Cart, Wishlist, About Us and Contact display on the right side of the header, left of the search box. The "Groups" button is not shown in the storefront header (groups are still reachable at `/groups`` and `/group/[slug]``).
- The admin "Category Order" panel (drag-and-drop) still controls display order of the category grid.

## Marketing hero (Original layout)

The hero section is fully admin-configurable from the Storefront panel and doubles as a conversion tool:

- Sale countdown timer (auto-hides when the offer ends).
- Rotating headline/accent/subtitle variants (6s cycle).
- "Chat on WhatsApp" CTA (uses the store's phone number).
- Payment & delivery trust strip (M-Pesa & cards, nationwide delivery, warranty).
- Featured products with "Sale" badges, star ratings, "Only N left" scarcity notes, and Ken Burns zoom.
- Category chips that auto-sync with the category list (deleted ones disappear, new ones appear).
- Live stats that count up on load; logged-in customers see a personalized greeting.
- Each booster has its own admin toggle (`showSearch``, `showTrustStrip``, `showWhatsApp``, etc.).
- **Theme-aware background** ? the hero background and text palette follow the visitor's device theme automatically: a light hero with dark text in light mode, the dark hero in dark mode. Admins can override either with custom color pickers (**Light theme background** / **Dark theme background**) in the hero form; leave them empty to keep following the device theme, and use "Reset to auto" to clear both. Text contrast is chosen automatically from the selected color (dark color ? white text, light color ? dark text).

## Theme

Dark mode by default with a light/dark toggle, persisted in `localStorage`` and applied via `[data-theme]`` CSS custom properties. All animations respect `prefers-reduced-motion``.

## Empty-cart animations

The empty cart page shows an animated SVG scene of the brand's gear-headed character leaning over the

cart and pointing into it while question marks rise out of the basket. In December a worried Santa gearhead (with an empty, drooping sack) joins it. Both are driven by CSS keyframes in `animations.css` (`EmptyCartAnimation.tsx`, `SantaGearAnimation.tsx`).

---

## 8. Backend API Summary

### Public (no auth)

- `GET /api/health` ? health check (usage stats only disclosed to the control plane).
- `GET /api/public-settings` ? store name, phone, email, currency, logo, Google Client ID, M-Pesa till.
- `GET /api/products`, `GET /api/products/:id`, `GET /api/products/:id/images`, `GET /api/products/:id/reviews`.
- `GET /api/categories`, `GET /api/subcategories`.
- `GET /api/plans`, `GET /api/layouts`, `GET /api/shipping/counties`, `GET /api/splashes`.
- `GET /api/shop/features` ? merged subscription + override features for UI gating.
- `GET /api/repairs-page` ? intro + service panels for the public `/repairs` page.

### Customer

- `POST /api/customer/login`, `POST /api/customer/google-login`.
- `GET/POST /api/orders`, `GET /api/orders/:id`, `GET /api/orders/:id/invoice`, `POST /api/orders/create-pending`, `PATCH /api/orders/:id`.
- `POST /api/cart`, `PATCH/DELETE /api/cart/:productId`.
- `POST /api/wishlist`, `POST /api/quotes/from-wishlist`.
- `POST /api/repairs`, `GET /api/repairs/mine`, `POST /api/repairs/mine/:id/message`, `POST /api/repairs/:id/quote-response`.
- `POST /api/products/:id/reviews`, `PUT/DELETE` review endpoints, `GET .../reviews/check`.
- `GET/POST /api/messages`.

### Staff

- `POST /api/auth/login`, `GET /api/auth/me`, `POST /api/auth/change-password`.
- Repair, stock, stock-transfer, stock-take, report, audit-log, and subscription endpoints (permission-gated).

### Admin

- Full CRUD for products, categories, staff, roles, plans, providers, orders, invoices, coupons, suppliers, branches, settings, splashes, storefront layouts, about-us content.

### Provider

- `GET /api/provider/orders`, `GET /api/provider/orders/:id`, `PATCH .../items/:itemId/cancel`, `PATCH .../status`.

### Control plane (authenticated with `x-control-plane-key`)

- `POST /api/admin/features/overrides` ? push per-client feature overrides.
- `PUT /api/plans/sync`, `GET /api/cloudinary-config`, `POST /api/control-plane/suspend|resume|redeploy`, `GET /api/control-plane/trigger-backup`.

---

## 9. How Data is Stored

- One **PostgreSQL** database per client (created on Neon by the control plane).
- Backend creates all tables on first boot from `server/schema.sql` and applies migrations.
- Settings are stored in a `settings` key/value table; the store name defaults to `STORE_NAME` env (or "Gear&Glitch").
- Images are stored on **Cloudinary** in production (per-client folder `gear-glitch/{client-slug}`), with optional base64 database backup via the `stored_images` table, and local disk in dev (`data/uploads/`).

---

## 10. Repair Workflow

### Customer side

1. Book a repair via `/repair-book`.
2. Backend creates a ticket.
3. Track ticket list in `/my-repairs`; view cost estimates with Accept/Decline and message the shop.

### Staff side

1. Manage tickets in `/admin` ? Services ? Repairs (assign technician, status, notes, parts, images).
2. Send cost estimates to customers; the customer's response updates the ticket.

---

## 11. Feature Flags & Per-Client Overrides

### Subscription plan features

Plans carry 51+ feature flags organized into 11 groups (Core Commerce, Inventory & Stock, Invoicing & Finance, Repairs & Service, Customer Engagement, WhatsApp & Communication, Multi-Location, Marketing & Storefront, Analytics & Security, Support & Account, Payments & Currency). Recent additions include Gift cards, Campaign pages, and Cart recovery. The frontend `useFeature()` hook shows/hides nav items and UI sections; server middleware enforces plan features on APIs.

### Control-plane overrides

From the control plane **Edit Client** modal, the operator can override a tenant's feature set without touching the plan:

- **Enabled** (blue) ? force-adds a feature the plan doesn't include.
- **Blocked** (red, struck through) ? hides a feature the plan normally includes.
- **Inherit** (neutral dash) ? falls back to the plan's defaults.
- Group **Select all** / **Block** buttons and a **Clear overrides** button.
- Saving pushes the overrides to the client backend, which merges them with plan features on `GET /api/shop/features`.

---

## 12. Store Identity (Tab Title, Branding)

- The browser tab title, `og:site\_name`, and `og:title` use the store's `settings.storeName`, falling back to a neutral "Welcome to our store" placeholder so no client ever displays another tenant's brand.
- `frontend/lib/app-context.tsx` sets `document.title` to the real store name once settings load.
- New clients provisioned by the control plane get `STORE\_NAME=<client name>` injected as a Render env var, and the backend seeds that name on first boot.
- Existing clients that were seeded before this fix should set their store name in **Admin ? Settings ? Store Info** once.

---

## 13. Control Plane

The control plane (`control-plane/`, port 4000, own Neon DB) is the operator hub:

- **Clients** ? provision (Neon + Render + Vercel), suspend/resume, redeploy, delete; health + uptime monitoring every 5 minutes; per-client feature overrides.
- **Plans** ? create/edit/activate/deactivate custom plans; sync to all clients.
- **Changelog** ? publish updates with email notifications to active clients.
- **Backups** ? `pg_dump` backups with UI download.
- **Settings** ? SMTP and Cloudinary configuration stored in DB.
- **Invoices** ? subscription invoicing: generate, **record payment** (one step: generate + mark paid + extend expiry + set next payment date + compute balance + auto-resume), view HTML/PDF, and email via the control plane's own SMTP.
- **Payment reminders** ? the Clients tab shows a banner with clients whose payment is due in 1 week / 2 days / today / overdue (driven by each client's `next_payment_date`).
- **Auto-deactivation** ? any active client whose subscription payment is missed past the `SUSPEND_GRACE_DAYS` grace period (default 3 days) is automatically **suspended** (never deleted) on each health cycle; recording a payment resumes it.
- **Notifications** ? a bell in the header lists payment due/overdue, client down, usage-over-limit, deploy/backup/provisioning-failed, and upgrade-request alerts with read state and deduplication.
- **Audit log** ? all admin actions recorded.
- **2FA** ? TOTP protection for operator accounts; API keys bypass 2FA.

### Provisioning (Add Client)

1. Create a Neon database.
2. Create a Render web service for the backend with env vars: `DATABASE_URL`, `JWT_SECRET`, `STORE_NAME`, `ADMIN_*`, `TECH_*`, `CONTROL_PLANE_SECRET`, Cloudinary.
3. Create a Vercel project for the frontend (`rootDirectory`frontend``, `BACKEND_URL`` set).
4. Set up Cloudflare DNS subdomain (optional).
5. Send a welcome email with credentials.

---

## 14. Useful Files

- `README.md` ? full feature overview, languages/technologies, and architecture.
- `LOCAL\_SETUP.md` ? local Windows setup guide.
- `DEPLOY\_CHECKLIST.md` ? production deployment checklist.
- `eTIMS\_INTEGRATION.md` ? KRA eTIMS compliance document.
- `ROADMAP.md` ? planned roadmap (payment collection, client portal, phone alerts, and more).
- `control-plane/README.md` ? control plane documentation (endpoints, env vars, provisioning).
- `server/index.ts` ? Express entry point (all routes).
- `server/db.ts` ? PostgreSQL layer and migrations.
- `frontend/lib/app-context.tsx` ? global context (auth, theme, settings, tab title).
- `frontend/layouts/original.tsx` ? premium marketing hero implementation.
- `control-plane/server/provision.ts` ? provisioning pipeline (Neon/Render/Vercel/Cloudflare).

---

## 15. Notes

- The app can be run locally with Node, a `.env` file, and a local PostgreSQL database.
- Backend must have `DATABASE\_URL` and a strong `JWT\_SECRET` to start.
- Staff tokens expire after 24h, customer tokens after 7 days.
- Production deployments: backend on Render, frontend on Vercel, databases on Neon.
- `npm run typecheck` runs TypeScript checks (root, `frontend/`, and `control-plane/` each have the script).

---

## 16. Recommended Next Steps

- Enable TypeScript `strict: true` and add ESLint/Prettier + a CI quality gate.
- Break up `server/index.ts` (~5,000 lines) and `frontend/pages/admin.tsx` into route/domain modules.
- Move to formal SQL migrations instead of imperative startup migrations.
- Enforce CSRF hard-fail and a strict CSP.
- Add end-to-end tests for provisioning, feature overrides, and layout switching.